NATURAL LANGUAGE PROCESSING · COURSE PROJECT

IMDB Sentiment Analysis

Classifying Movie Reviews as Positive or Negative Using NLP

Team Members

1- Youssef emad samouel

2-Nada Mahmoud ahmed

3-Radwa Abdulrahman Ahmed

4-Reham Hamdi Abdel Sattar

5-Raghda Badran Hassan Abdel

6-Shahd Amr Shadeed

Table of Contents

1. Project Overview
2. Objectives
3. Dataset
4. Text Preprocessing Pipeline
   - 4.1 Text Cleaning
   - 4.2 Tokenisation
   - 4.3 Stopword Removal
   - 4.4 Lemmatisation
5. Feature Extraction
6. Classification Model
7. Training & Evaluation
8. Results
9. Key Design Decisions
10. Tools & Technologies
11. Limitations & Future Work
12. Conclusion

1. Project Overview

This project addresses one of the most practical problems in Natural Language Processing: **sentiment analysis**. Given a movie review written by a real viewer, the goal is to automatically determine whether that review expresses a **positive** or **negative** opinion — a task known as binary sentiment classification.

Sentiment analysis has broad real-world applications: businesses use it to monitor customer feedback at scale, streaming platforms use it to understand audience reception, and researchers use it to study public opinion trends. By building this classifier from the ground up on a large, real-world dataset, the project demonstrates the complete lifecycle of an NLP solution — from raw noisy text all the way to a working, evaluatable prediction system.

The project uses the **IMDB Large Movie Review Dataset**, a widely used academic benchmark containing nearly 50,000 labelled reviews. The system applies a classical NLP preprocessing pipeline to clean and normalise the text, extracts numerical features using TF-IDF vectorisation, and trains a Logistic Regression classifier to distinguish positive from negative sentiment. The entire workflow runs in **Google Colab** with data stored on Google Drive.

Core Idea	The project treats a movie review as a bag of weighted words. By learning which words and phrases appear most often in positive versus negative reviews, the model can predict sentiment on reviews it has never seen before.

2. Objectives

The project was designed to achieve the following goals:

1. **Build a complete NLP pipeline**
   Design and implement every stage of text preprocessing from scratch — cleaning, tokenisation, stopword removal, and lemmatisation — to understand how raw text is transformed into machine-readable input.

2. **Apply TF-IDF feature extraction**
   Convert processed text into a numerical representation that captures how important each word or phrase is within a review relative to the entire corpus.

3. **Train a sentiment classifier**
   Use Logistic Regression to learn the relationship between TF-IDF features and sentiment labels, producing a model that generalises to unseen reviews.

4. **Evaluate model performance rigorously**
   Measure accuracy, precision, recall, and F1-score on a held-out test set, and visualise results through a confusion matrix heatmap.

5. **Demonstrate end-to-end inference**
   Show that the trained system can accept any raw review text and return a sentiment prediction in real time.

3. Dataset

The project uses the **IMDB Large Movie Review Dataset**, one of the most well-established benchmarks in sentiment analysis research. The dataset was sourced from Kaggle in CSV format and stored on Google Drive, where the notebook automatically discovers and loads it at runtime.

The working dataset contains **49,396 reviews** after the initial cleaning pass (slightly fewer than the nominal 50,000 due to removal of duplicates and malformed entries). Each review is a plain-text paragraph or series of paragraphs written by a real IMDB user, and each is labelled either **positive** or **negative** based on the star rating the user gave.

Property	Detail
Total reviews	49,396
Positive reviews	24,698 (50%)
Negative reviews	24,698 (50%)
Features	Review text + Sentiment label
Label encoding	Positive = 1 · Negative = 0
Source	Kaggle — IMDB Dataset (CSV)
Storage	Google Drive, auto-discovered by filename

3.1 Why This Dataset?

The IMDB dataset is ideal for this project for several reasons. First, it is **perfectly balanced** — exactly half positive and half negative — which means the model cannot achieve good performance simply by always predicting the majority class. A naive classifier that always predicts "positive" would only be correct 50% of the time, so any meaningful model must learn genuine linguistic patterns.

Second, the reviews are **long-form and noisy**. Real IMDB reviews contain HTML tags (like <br /> line breaks), URLs, numbers, unusual punctuation, and highly variable length — from a single sentence to several paragraphs. This makes the preprocessing challenge realistic and non-trivial.

Third, the dataset is **widely benchmarked**, meaning the project's results can be meaningfully compared against published baselines from the academic literature.

4. Text Preprocessing Pipeline

Raw movie reviews cannot be fed directly into a machine learning model. They contain HTML markup, inconsistent capitalisation, numbers, punctuation, and filler words that carry no sentiment signal. The preprocessing pipeline transforms each review through four sequential stages, each applied to every review in the dataset:

1. **Text Cleaning**
   Remove HTML tags, normalise case, strip URLs, digits, and punctuation
2. **Tokenisation**
   Split each cleaned review into individual word tokens
3. **Stopword Removal**
   Discard common function words that carry no sentiment signal
4. **Lemmatisation**
   Reduce each token to its dictionary base form

Pipeline Output After all four stages, a review that began as a paragraph of noisy HTML-embedded text becomes a clean list of normalised, meaningful words. These are then joined back into a string and passed to the TF-IDF vectoriser.

4.1 Text Cleaning

The first stage scrubs raw review text of everything that is not meaningful English prose. Reviews on IMDB are entered through a web form, so they frequently contain HTML line-break tags such as <br /> and other markup that would otherwise be treated as vocabulary. These are stripped out first.

The entire text is then converted to lowercase so that "Movie", "movie", and "MOVIE" are treated as the same word. Any URLs are removed, since web addresses carry no sentiment. Digits are stripped — numbers like "2001" or "10" do not express opinion. Non-letter characters (punctuation, special symbols) are removed, and any runs of extra whitespace are collapsed into single spaces. The result is a clean, lowercase string of plain English words.

Example Before cleaning: "$25,000 Pyramid Clues: Deep Blue Sea. Tremors.

Movies that were funnier and more thrilling than Snakes on a Plane." → After cleaning: "pyramid clues deep blue sea tremors movies that were funnier and more thrilling than snakes on a plane"

4.2 Tokenisation

Once the text is clean, it must be broken into individual units that the model can work with. **Tokenisation** is the process of splitting a string into a list of individual words — called tokens.

The project uses NLTK's **word_tokenize()** function, which is more linguistically aware than a simple split on spaces. It correctly handles edge cases such as contractions ("don't" becomes "do" and "n't"), and it separates punctuation that may still be attached to words after cleaning. The output of this stage is a Python list of individual word strings for each review.

4.3 Stopword Removal

Stopwords are extremely common words like "the", "a", "is", "in", and "of" that appear in virtually every document regardless of topic or sentiment. Because they appear everywhere equally, they carry almost no information about whether a review is positive or negative. Including them would bloat the vocabulary, slow down training, and potentially dilute the signal from genuinely meaningful words.

The project uses the standard English stopword list from the NLTK library and removes any token that appears on that list. However, a critical exception is made: **nine negation words are preserved** — "not", "no", "nor", "never", "n't", "neither", "nobody", "nothing", and "nowhere". These words are technically stopwords, but they fundamentally reverse the meaning of the words around them. Removing "not" would cause "not good" to be processed as just "good", completely inverting the sentiment of that phrase.

Impact	Stopword removal reduces the average token count per review by roughly 40–50%, dramatically shrinking the vocabulary that TF-IDF must represent — without losing any words that actually carry sentiment.

4.4 Lemmatisation

English words appear in many different forms depending on tense, number, and other grammatical features. "Run", "runs", "running", and "ran" all refer to the same concept. Without normalisation, the model would treat these as four different vocabulary items and learn separate (and weaker) associations for each.

Lemmatisation solves this by reducing each word to its canonical dictionary form — called its *lemma*. For example, "running" becomes "run", "movies" becomes "movie", and "better" becomes "good". The project uses NLTK's **WordNetLemmatizer**, which looks up words in the WordNet lexical database to find the correct base form.

Lemmatisation was chosen over the simpler alternative — **stemming** — deliberately. Stemming (e.g. PorterStemmer) is faster but works by mechanically chopping suffixes, often producing non-words: "running" becomes "runn", "happiness" becomes "happi". Lemmatisation produces real dictionary

words, which makes the TF-IDF vocabulary more interpretable and the features more meaningful.

5. Feature Extraction — TF-IDF

After preprocessing, each review is a clean string of normalised words. But machine learning models cannot work with raw text — they require numbers. The next step is to convert each review into a numerical vector that captures which words it contains and how important those words are.

The project uses **TF-IDF** (Term Frequency–Inverse Document Frequency) vectorisation, implemented through scikit-learn's TfidfVectorizer. TF-IDF assigns each word in a review a weight that reflects two things simultaneously:

- **Term Frequency (TF)** — how often the word appears in this specific review. A word that appears many times in a review is likely important to its meaning.
- **Inverse Document Frequency (IDF)** — how rare the word is across the entire dataset. A word that appears in almost every review (like "film") carries less discriminative power than a word that appears in only a few reviews.

The product of these two values gives a high weight to words that are frequent in a particular review but rare across the corpus — exactly the words most likely to be distinctive and meaningful for that review's sentiment.

Setting	Value	Explanation
Vocabulary size	5,000 terms	The 5,000 most discriminative words retained
N-gram range	Unigrams + Bigrams	Both single words and two-word phrases are captured
Output	Sparse matrix	49,396 reviews x 5,000 features
Fitted on	Full dataset	Before the train/test split

Two configuration choices deserve explanation. The vocabulary is capped at **5,000 features** to keep the model tractable — including every word in the corpus would create an enormous, sparse matrix full of noise. The 5,000 most discriminative terms are retained, which empirical research on this dataset suggests is sufficient for strong baseline performance.

The inclusion of **bigrams** (two-word phrases) alongside individual words is a significant design choice. Many sentiment-bearing expressions consist of multiple words: "highly recommend", "total waste", "not bad", "brilliantly acted". Treating these as single units allows the model to capture compositional meaning that would be lost if only individual words were considered.

6. Classification Model

With each review represented as a 5,000-dimensional TF-IDF vector, the next step is to train a model that learns which patterns in those vectors correspond to positive versus negative sentiment. The project uses **Logistic Regression**, the standard baseline classifier for text classification tasks.

Logistic Regression works by learning a weight for every feature in the input — in this case, a weight for each of the 5,000 TF-IDF vocabulary terms. A word with a high positive weight pushes the model toward predicting "positive sentiment"; a word with a high negative weight pushes it toward "negative sentiment". During inference, the model computes a weighted sum of the input features and passes it through a sigmoid function to produce a probability between 0 and 1. If this probability exceeds 0.5, the review is classified as positive; otherwise, negative.

Why Logistic Regression? Logistic Regression is especially well-suited to TF-IDF feature spaces because these spaces are very high-dimensional but sparse — most reviews contain only a small fraction of the 5,000 vocabulary terms. Logistic Regression handles this efficiently and is highly interpretable: the learned weights directly reveal which words the model associates with positive or negative sentiment.

The model is configured with **L2 regularisation** (the default in scikit-learn), which penalises large weights and prevents overfitting. This is important because with 5,000 features and ~39,000 training samples, the model has enough capacity to memorise the training data if unconstrained. The solver is set to **L-BFGS** with a maximum of 1,000 iterations to ensure full convergence.

Parameter	Configuration
Model type	Logistic Regression (binary classification)
Regularisation	L2 (Ridge) — prevents overfitting
Solver	L-BFGS — efficient for dense, high-dimensional data
Max iterations	1,000 — ensures convergence
Decision boundary	Probability threshold of 0.5
Output	Binary label: 0 (Negative) or 1 (Positive)

7. Training & Evaluation

7.1 Train / Test Split

Before training, the full TF-IDF feature matrix is divided into two non-overlapping subsets. **80% of the reviews** (~39,516) form the training set, which the model sees during learning. The remaining **20%** (~9,880 reviews) are held out as the test set and are never used during training — they exist solely to measure how well the trained model generalises to new, unseen data.

The split is performed with a fixed **random seed (42)**, which guarantees that every run of the notebook produces the exact same partition. This is essential for reproducibility: anyone running the notebook will see the same results. The split preserves the 50/50 class balance in both subsets through stratification, ensuring neither subset is accidentally skewed toward one sentiment class.

Subset	Size	Purpose
Training set	~39,516 reviews (80%)	Used to fit the model weights
Test set	~9,880 reviews (20%)	Used only for final evaluation

7.2 Evaluation Strategy

Once trained, the model's performance is measured on the held-out test set using four complementary metrics. Relying on accuracy alone can be misleading — for example, on an imbalanced dataset a model could score 90% accuracy simply by always predicting the majority class. Since this dataset is balanced, accuracy is meaningful here, but the additional metrics provide a more complete picture:

Metric	What It Measures	Why It Matters
Accuracy	The proportion of all test reviews classified correctly.	Overall correctness of the model.
Precision	Of all reviews the model labels as "positive", what fraction truly are?	How reliable are the positive predictions?
Recall	Of all genuinely positive reviews, what fraction does the model correctly catch?	How many positives does the model miss?
F1-Score	The harmonic mean of precision and recall.	Balanced measure when precision and recall trade off.

In addition to these four numeric metrics, a **confusion matrix** is produced and visualised as a heatmap. The confusion matrix shows, in a grid, how many reviews from each true class were assigned to each predicted class — revealing not just the overall error rate, but whether errors are skewed (e.g. the model misclassifying negatives as positives more often than the reverse).

8. Results

The Logistic Regression classifier achieves strong performance on the held-out test set, demonstrating that the classical NLP pipeline combined with TF-IDF features is highly effective for this task. The model reaches approximately **89% test accuracy**, meaning it correctly classifies roughly 9 out of every 10 unseen reviews.

Class	Precision	Recall	F1-Score	Support
Negative	~0.89	~0.89	~0.89	~4,940
Positive	~0.89	~0.89	~0.89	~4,940
Weighted Average	**~0.89**	**~0.89**	**~0.89**	**~9,880**

The results are symmetric across both classes, reflecting the balanced nature of the dataset. The model performs equally well at identifying positive reviews and negative reviews — it does not exhibit a bias toward predicting one class more than the other. Precision and recall are both at approximately 89%, indicating that the model is both reliable when it makes a prediction and thorough in catching reviews of each sentiment type.

Context An ~89% accuracy on a balanced 50/50 dataset using only classical NLP features is a strong result. Published Logistic Regression baselines on IMDB consistently fall in the 88–91% range, confirming the pipeline is well-implemented. State-of-the-art transformer models (BERT, RoBERTa) reach 95–96%, but require orders of magnitude more compute.

8.1 Sample Predictions

The trained pipeline includes an end-to-end predict_sentiment() function that accepts any raw review text, runs it through the full preprocessing and vectorisation pipeline, and returns a sentiment label. The table below shows illustrative examples:

Review (sample)	Prediction	Explanation
"I absolutely loved this movie! The acting was superb."	Positive	Strongly positive vocabulary correctly identified.
"Terrible film. The plot was predictable and completely forced."	Negative	Explicitly negative words dominate the TF-IDF signal.
"Not the worst film I have seen, but far from great."	Negative	Negation preservation allows "not ... great" to be correctly read as negative.

Review (sample)	Prediction	Explanation
"A masterpiece of storytelling. Highly recommended."	Positive	High-weight positive bigrams ("highly recommend") drive the prediction.

9. Key Design Decisions

Several design choices were made deliberately during the project, each with a concrete rationale.

Preserving Negation Words	Standard NLP practice removes all stopwords, including "not", "no", and "never". For general text classification tasks this is reasonable, but for sentiment analysis it is actively harmful. These words reverse the polarity of surrounding terms: "not good" means the opposite of "good", "never boring" is a strong positive. The project explicitly removes nine negation terms from the stopword list before filtering, preserving their contribution to the TF-IDF features.
Lemmatisation over Stemming	Both lemmatisation and stemming normalise word forms, but they do so differently. Stemming is faster — it mechanically strips suffixes — but produces non-words that reduce the quality of the TF-IDF vocabulary. Lemmatisation looks up the correct base form in a dictionary, producing valid English words. This was chosen to keep the feature space interpretable and linguistically meaningful.
Bigrams in TF-IDF	Using only single words (unigrams) would miss many sentiment-bearing expressions that only make sense as phrases: "highly recommend", "total disaster", "not bad". Including bigrams in the vectoriser doubles the vocabulary candidates before the 5,000-feature cap, allowing the model to learn from multi-word sentiment signals.
5,000 Feature Cap	Capping the TF-IDF vocabulary at 5,000 terms is a practical tradeoff between expressiveness and tractability. Using too few features loses discriminative vocabulary; using too many introduces rare, noisy terms and increases computation. Research on this dataset consistently shows diminishing returns past 10,000 features for classical models, making 5,000 an efficient choice.
Logistic Regression as the Model	A neural network or ensemble model would likely achieve higher accuracy, but Logistic Regression was chosen as the primary model because it is interpretable, fast to train, and establishes a rigorous baseline. Its learned weights directly expose which words most strongly predict each sentiment class, providing insight into what the model has actually learned from the data.

10. Tools & Technologies

The project is built entirely using open-source Python libraries, all of which are pre-installed in Google Colab. No paid tools, APIs, or cloud compute credits are required.

Library / Tool	Role in the Project
Python 3	Core programming language
Google Colab	Cloud-hosted Jupyter notebook environment (free GPU/CPU runtime)
Google Drive	Dataset storage — auto-mounted and auto-discovered at runtime
NLTK	NLP toolkit: tokeniser, stopword lists, WordNet lemmatiser
scikit-learn	TF-IDF vectoriser, Logistic Regression, train/test split, evaluation metrics
pandas	DataFrame management for reviews, labels, and processed columns
NumPy	Underlying numerical array operations
Matplotlib	Token distribution histograms and word-frequency bar charts
Seaborn	Confusion matrix heatmap visualisation
re (stdlib)	Regular expressions for the text cleaning pipeline
collections	Counter for top-word frequency analysis

11. Limitations & Future Work

11.1 Current Limitations

Like any first-pass NLP system, this project has several deliberate simplifications that represent honest tradeoffs between complexity and clarity:

- **Binary classification only.** The model can only output "positive" or "negative". Real sentiment is a spectrum — a review might be mixed, sarcastic, or express nuanced emotions that don't fit neatly into two categories.
- **Domain-specific training.** The classifier was trained exclusively on movie reviews. Applied to product reviews, social media posts, or news comments, it may perform significantly worse because the vocabulary and writing style differ.
- **No cross-validation.** A single 80/20 split is used for evaluation. K-fold cross-validation across multiple splits would give a more statistically robust estimate of generalisation performance.
- **Bag-of-words assumption.** TF-IDF treats each review as an unordered set of terms. Word order and long-range context — "the beginning was brilliant but the ending ruined everything" — are mostly lost, even with bigrams.
- **No handling of sarcasm or irony.** A review like "Oh brilliant, yet another predictable sequel" uses positive words sarcastically. The model would likely misclassify this as positive.

11.2 Possible Extensions

The following improvements could meaningfully advance the project beyond its current scope:

- **Transformer-based models.** Fine-tuning BERT, DistilBERT, or RoBERTa on IMDB would likely push accuracy above 95%, by leveraging deep contextual representations that understand word meaning in context.
- **K-fold cross-validation.** Running the full training and evaluation cycle across five or ten different train/test splits and averaging the results would produce more reliable performance estimates.
- **Multi-class sentiment.** Rather than binary positive/negative, predict a star rating (1–5) to capture the spectrum of opinion.
- **Out-of-domain evaluation.** Test the trained model on Amazon product reviews or Twitter data to measure how well it generalises beyond movie reviews.
- **Deployment as a web application.** Wrap the inference pipeline in a Gradio or Flask app so users can enter any review text and receive a real-time sentiment prediction.

12. Conclusion

This project successfully demonstrates how a carefully designed classical NLP pipeline can transform raw, noisy movie review text into an accurate sentiment classifier. Starting from HTML-embedded user reviews and ending with a working prediction system that achieves ~89% accuracy on nearly 10,000 unseen reviews, the project covers the full arc of a real-world NLP workflow.

The preprocessing pipeline — cleaning, tokenisation, stopword removal with negation preservation, and lemmatisation — illustrates that the quality of input representations is often more important than the sophistication of the model. TF-IDF with bigrams captures meaningful compositional sentiment signals that a simple bag-of-words approach would miss, and Logistic Regression provides a fast, interpretable, and competitive classifier on the resulting feature space.

The design decisions made throughout — preserving negation words, choosing lemmatisation over stemming, including bigrams, capping features at 5,000 — each reflects a concrete understanding of the linguistic and statistical properties of the problem. These choices, and their rationale, are as valuable as the final accuracy figure.

Summary The ~89% accuracy achieved here using only classical methods and standard libraries establishes a solid, reproducible baseline. Future work can build on this foundation by introducing contextual embeddings, cross-validation, and multi-domain evaluation.

IMDB Sentiment Analysis — Project Documentation · Version 1.0 · May 09, 2026